Artificial Intelligence

Informed (Heuristic) Search Strategies

Chiến lược tìm kiếm kinh nghiệm



Tìm kiếm kinh nghiệm

- Phương pháp tìm kiếm mù có khả năng thám hiểm không gian trang thái để tìm ra trạng thái đích, tuy nhiên nó rất là không hiệu quả trong hầu hết bài toán
- Với sự trợ giúp của kinh nghiệm, trí thức đặc trưng của vấn đề, phương pháp tìm kiệm kinh nghiệm rất là hiệu quả
- Là phương pháp mà sử dụng hàm đánh giá cung cấp trí thức đặc trưng về bài toán để hướng dẫn sự tìm kiếm

Nội dung

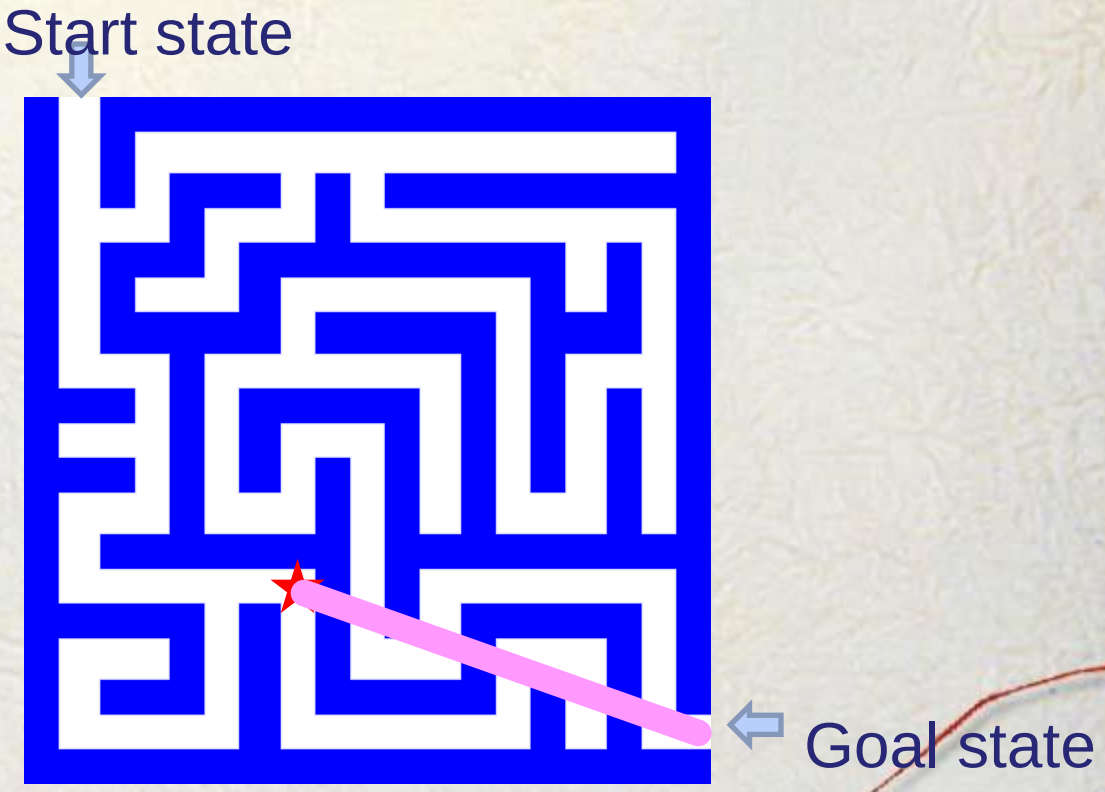
- Hàm đánh giá
- Phương pháp tìm kiếm kinh nghiệm
  - Greedy Best-first search
  - Beam Search
  - Hill-climbing Search
  - A* search

Heuristics (Kinh nghiệm)

- “Heuristics are criteria, methods or principles for deciding which among several alternative courses of action promises to be the most effective in order to achieve some goal.”
- Can make use of heuristics in deciding which is the most “promising” path to take during search.
- Evaluation function h(u): a measure to evaluate the *distance* of state u from the goal. e.g: h(u) = 0 if u is the goal state.
- Evaluation functions (or heuristic functions) are problem specific functions that provide an estimate of solution cost.

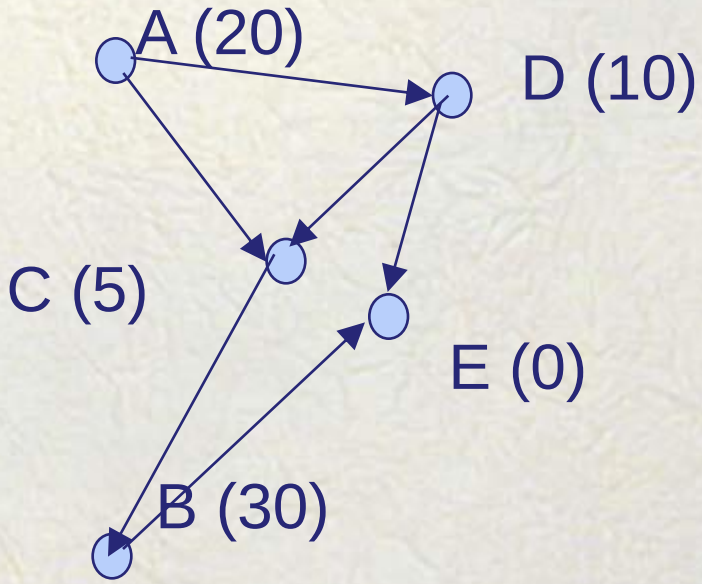
Heuristic function (Hàm đánh giá)

- **Heuristic function** $h(n)$ estimates the cost of reaching goal from node n
- Ví dụ:

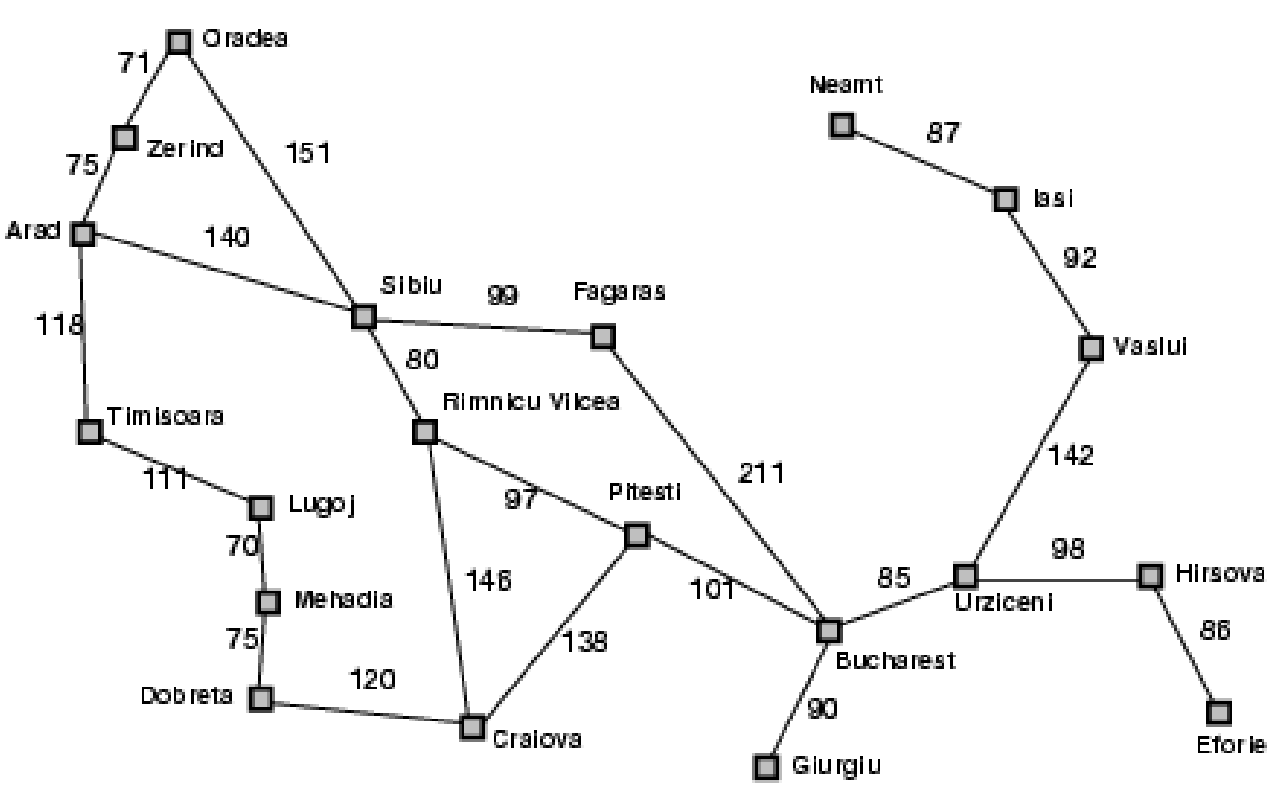


Hàm đánh giá

- Bài toán du lịch: lấy độ dài theo đường chim bay từ một thành phố đến thành phố đích làm giá trị của hàm đánh giá



Hàm đánh giá



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Hàm đánh giá

- Từ một vị trí, không biết bản đồ xung quanh, hãy đến điểm cao nhất trong khu vực – ???
- Tìm đường trên phố để đến được tòa nhà cao nhất thành phố - ???

Hàm đánh giá

- Từ một vị trí, không biết bản đồ xung quanh, hãy đến điểm cao nhất trong khu vực – **hàm đánh giá chính là độ cao tại mỗi điểm**
- Tìm đường trên phố để đến được tòa nhà cao nhất thành phố **- hàm đánh giá là khoảng cách từ mỗi điểm theo đường chim bay đến tòa nhà**

Hàm đánh giá

Bài toán 8 số:

- Số quân nằm không đúng vị trí, hoặc
- Tổng số khoảng cách vị trí của các quân và vị trí đích của chúng

4	3	1
	6	5
8	2	7

1	2	3
8		4
7	6	5

Hàm đánh giá

- Số quân nằm không đúng vị trí: 8

- Tổng số khoảng cách vị trí của các quân và vị trí đích của chúng: 3 + 1 + 2 + 1 + 1 + 1 + 1 + 2 + 2 = 14

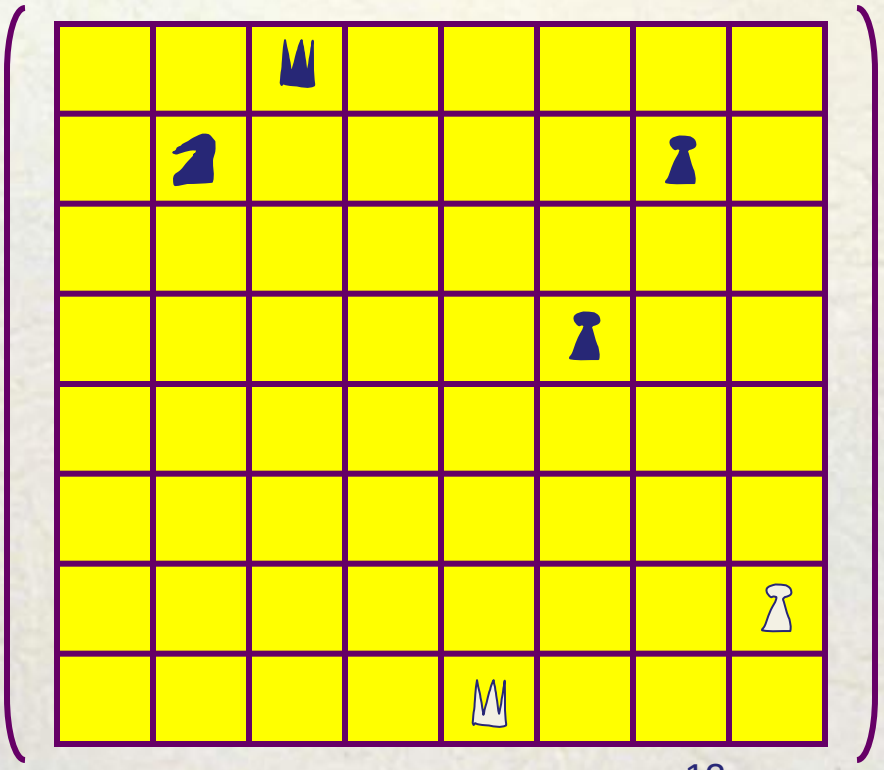
4	3	1
	6	5
8	2	7

1	2	3
8		4
7	6	5

Hàm đánh giá

- F(T) = (Value count of black pieces) - (Value count of white pieces)

f



= v( ) + v( )
+ v( ) + v( )
- v( ) - v( )

Hàm đánh giá

- Có thể có nhiều cách đánh giá
- Các hàm đánh giá có thể không phải hàm tối ưu
- Cách chọn hàm đánh giá quyết định rất nhiều kết quả của các kỹ thuật tìm kiếm kinh nghiệm

Tìm kiếm kinh nghiệm

Các giai đoạn:

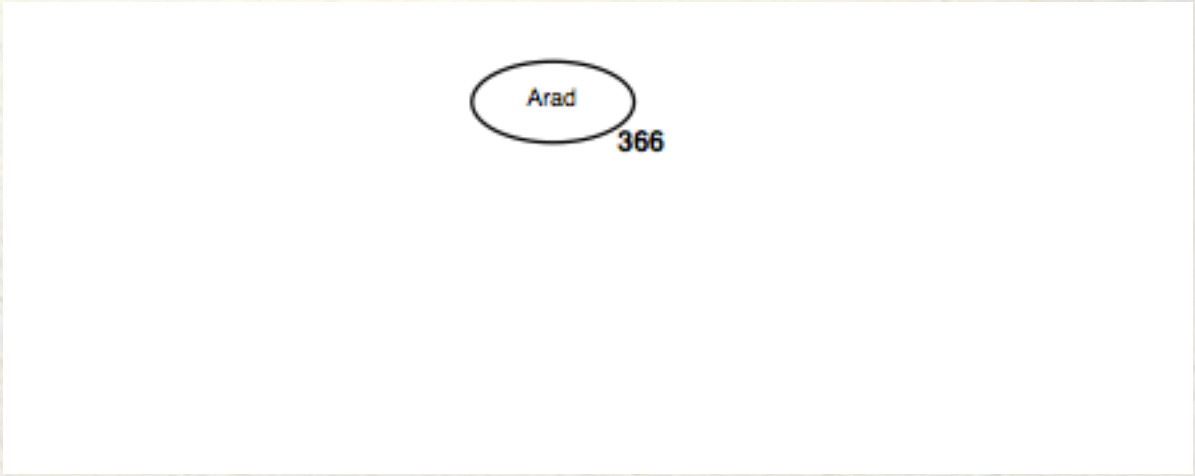
1. Tìm biểu diễn thích hợp mô tả các trạng thái và các toán tử của vấn đề
2. Xây dựng hàm đánh giá
3. Thiết kế chiến lược chọn trạng thái để phát triển ở mỗi bước

Có một lớp các thuật toán tìm kiếm tốt nhất đầu tiên (Best First Search) sử dụng các hàm đánh giá khác nhau

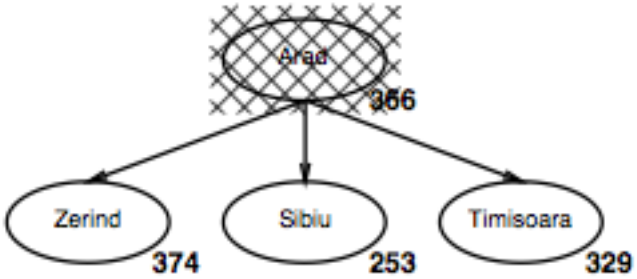
Tìm kiếm tốt nhất đầu tiên

- Tìm kiếm tốt nhất đầu tiên = Tìm kiếm bề rộng + Hàm đánh giá
- Khác với tìm kiếm theo chiều rộng, tìm kiếm tốt nhất đầu tiên chọn đỉnh để phát triển là đỉnh tốt nhất được xác định bởi hàm đánh giá
- Đỉnh tốt nhất là đỉnh có giá trị hàm đánh giá là nhỏ nhất, đỉnh này có thể ở mức hiện tại hoặc ở các mức trên

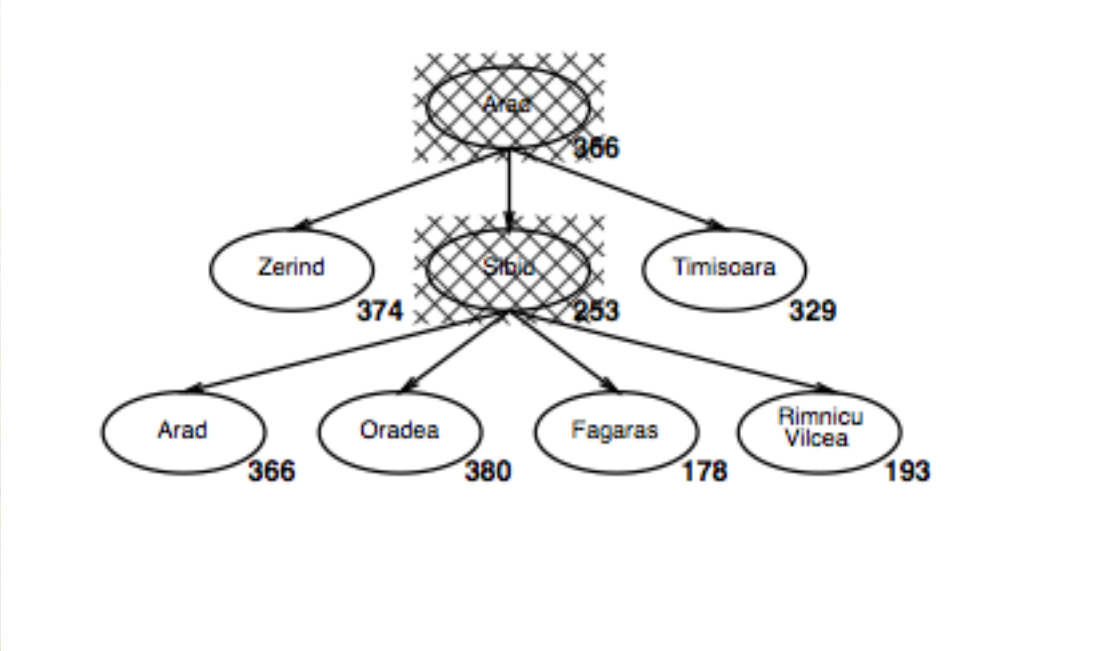
Tìm kiếm tốt nhất đầu tiên (ví dụ)



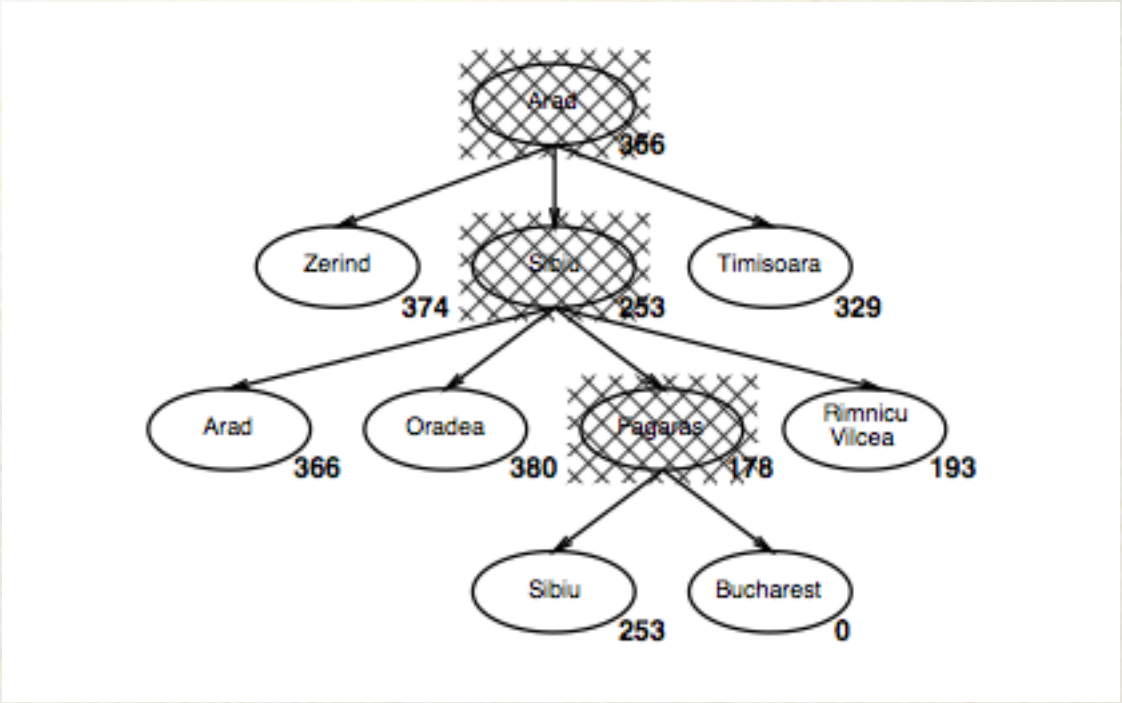
Ví dụ



Ví dụ


Arad
366
Zerind
374
Sibiu
253
Timisoara
329
Arad
366
Oradea
380
Fagaras
178
Rimnicu
Vilcea
193

Ví dụ

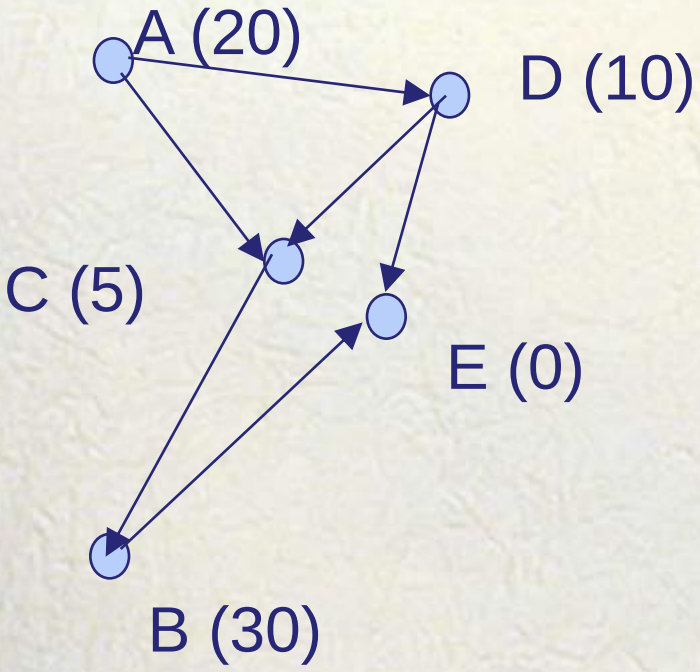

Arad
366
Zerind
374
Sibiu
253
Timisoara
329
Arad
366
Oradea
380
Pagaras
178
Rimnicu Vilcea
193
Siblu
253
Bucharest
0

Tìm kiếm tốt nhất đầu tiên

1. Khởi tạo danh sách **L** chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu danh sách **L**;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {Đặt v vào danh sách **L** sao cho **L** được sắp xếp thep thứ tự tốt đến xấu của hàm đánh giá;}

Tìm kiếm tốt nhất đầu tiên

Tìm đường từ A đến E



- Tìm E

- **L**: A - xét A
- **L**: C, D - xét C
- **L**: D, B - xét D
- **L**: E, B - xét E
- Tìm thấy E

Đặc trưng của tìm kiếm tốt nhất đầu tiên

- Complete? **No** – can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →

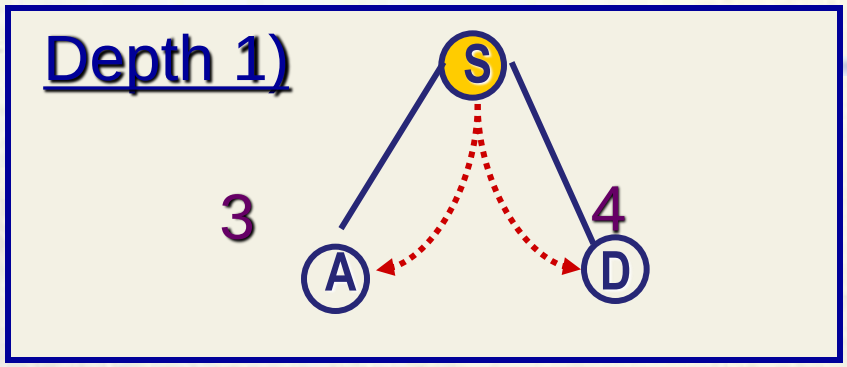
  Complete in finite space with repeated-state checking
- Time? $O(b^m)$, m is the maximum depth in search space
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? **No**

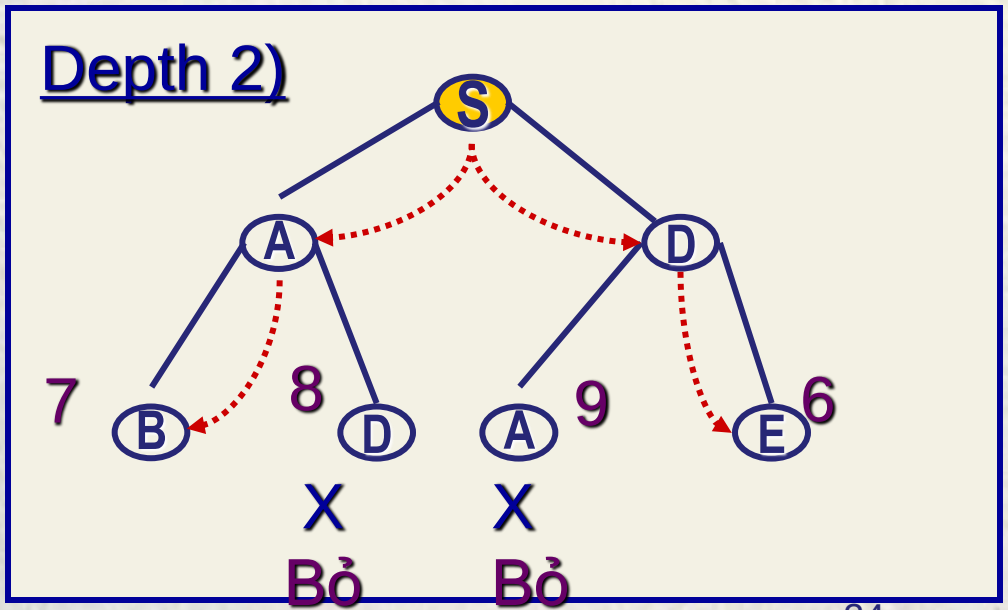
Một hàm đánh giá tốt có thể giảm thời gian và không gian nhớ một cách đánh kể

Beam Search (Tìm kiếm Beam)

- Tương tự như tìm kiếm tốt nhất đầu tiên
- Tuy nhiên chỉ phát triển k đỉnh ở mức tiếp theo chứ không phát triển toàn bộ
- Ưu điểm: độ phức tạp tính toán tốt hơn
- Nhược điểm: không tìm kiếm toàn bộ, nên có thể không tìm thấy đỉnh tối ưu nhất

Beam search (1):



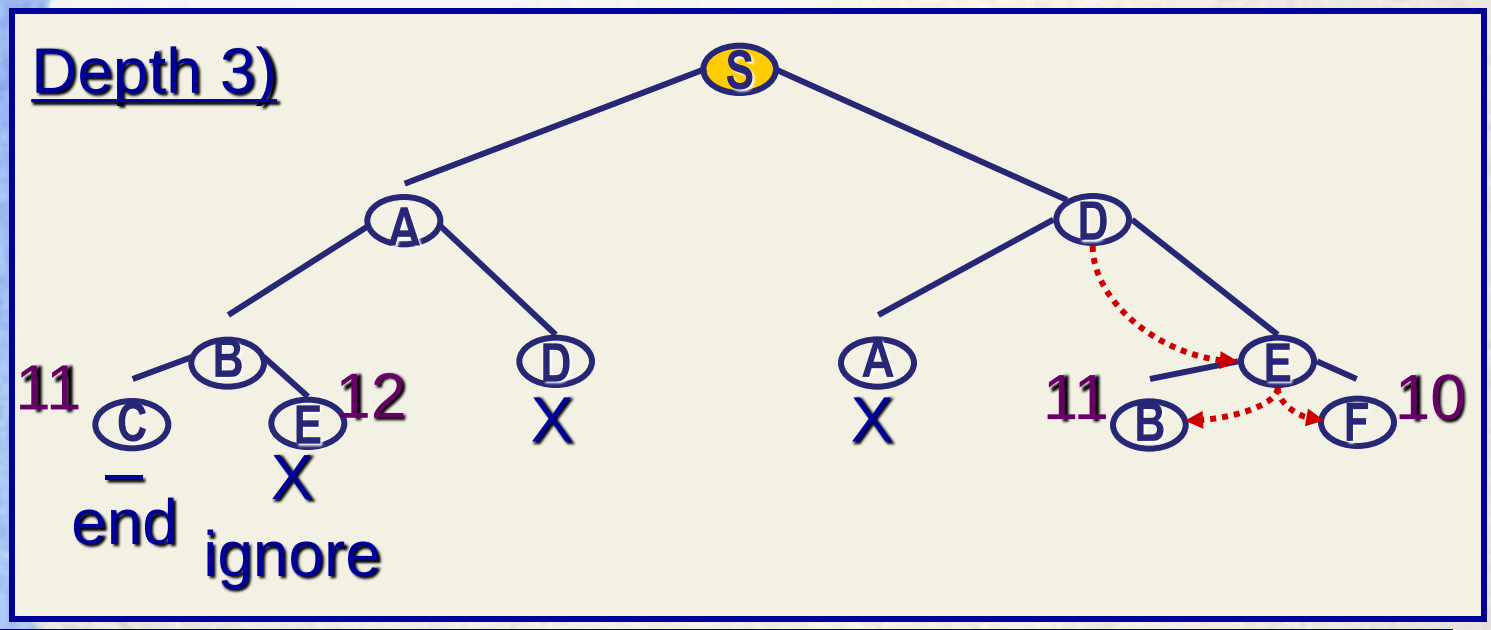


- Assume a pre-fixed WIDTH (example : 2)
- Perform breadth-first, BUT:
- Only keep the WIDTH best new nodes

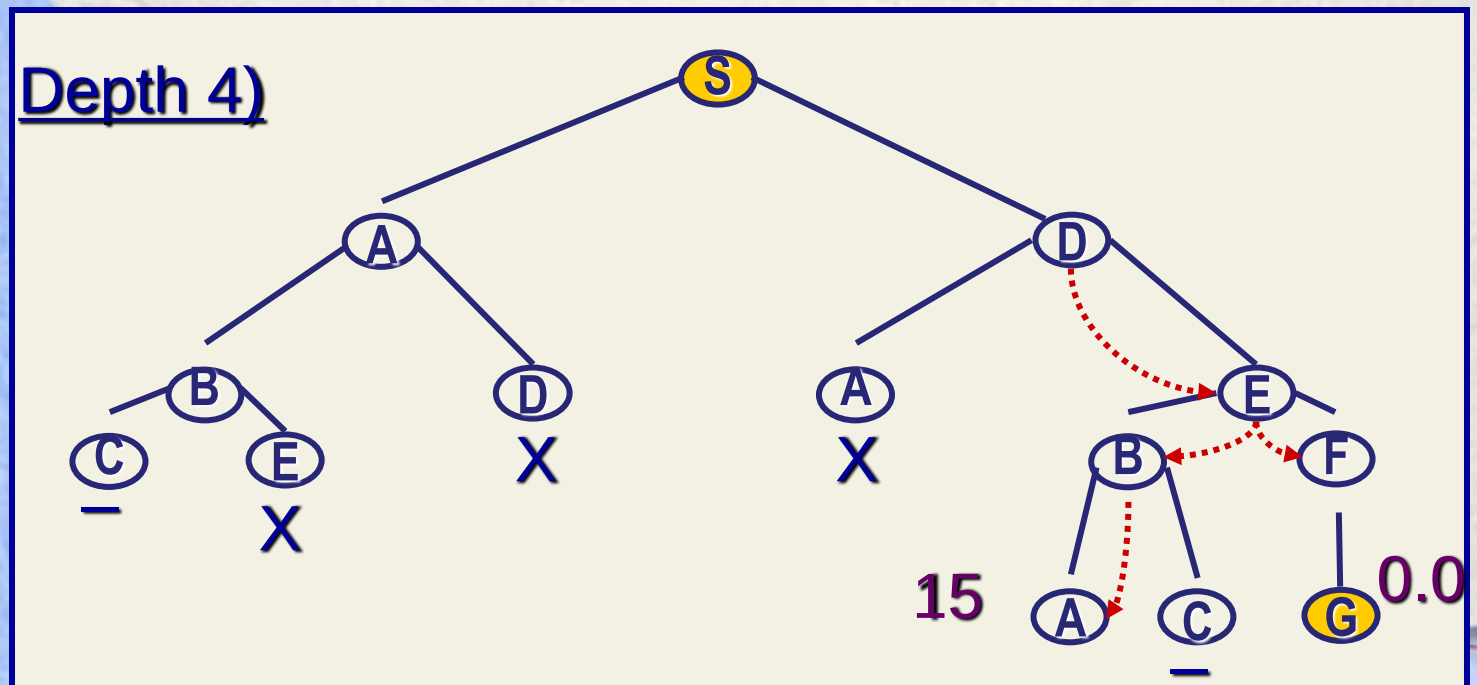
  depending on heuristic

  at each new level.

Beam search (2):



- Optimi-zation: ignore leafs that are not goal nodes (see C)



Beam search algorithm:

- Xem như bài tập về nhà

Properties:

◎ Completeness:

➔ Hill climbing: YES (backtracking), Beam search: NO

◎ Speed/Memory:

➔ Hill climbing:

◆ same as Depth-first (in worst case)

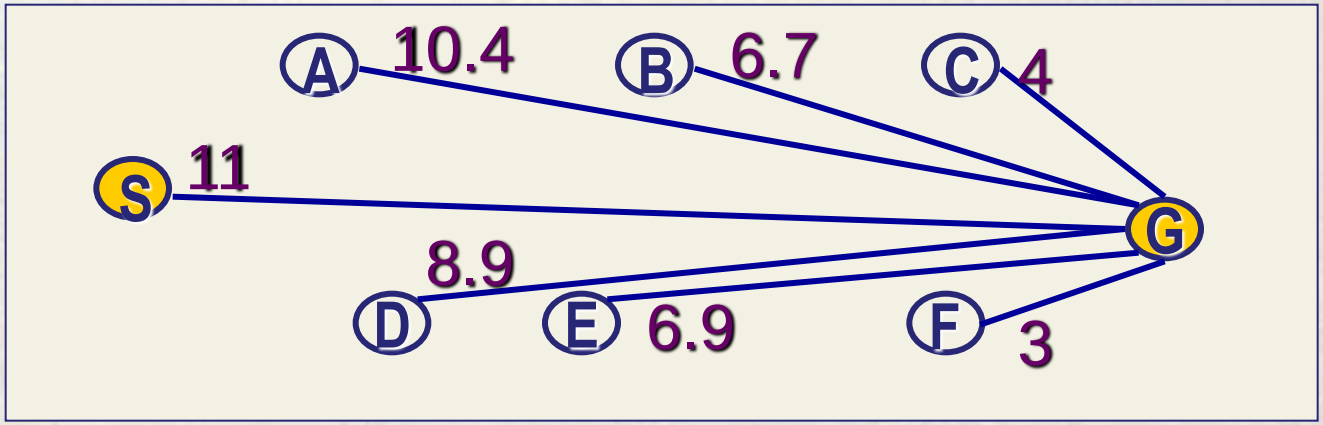
➔ Beam search:

◆ QUEUE always has length WIDTH, so memory usage is constant = WIDTH, time is of the order of WIDTH*d*b or WIDTH*m*b if no solution is found

Tìm kiếm leo đồi (Hill-climbing Search)

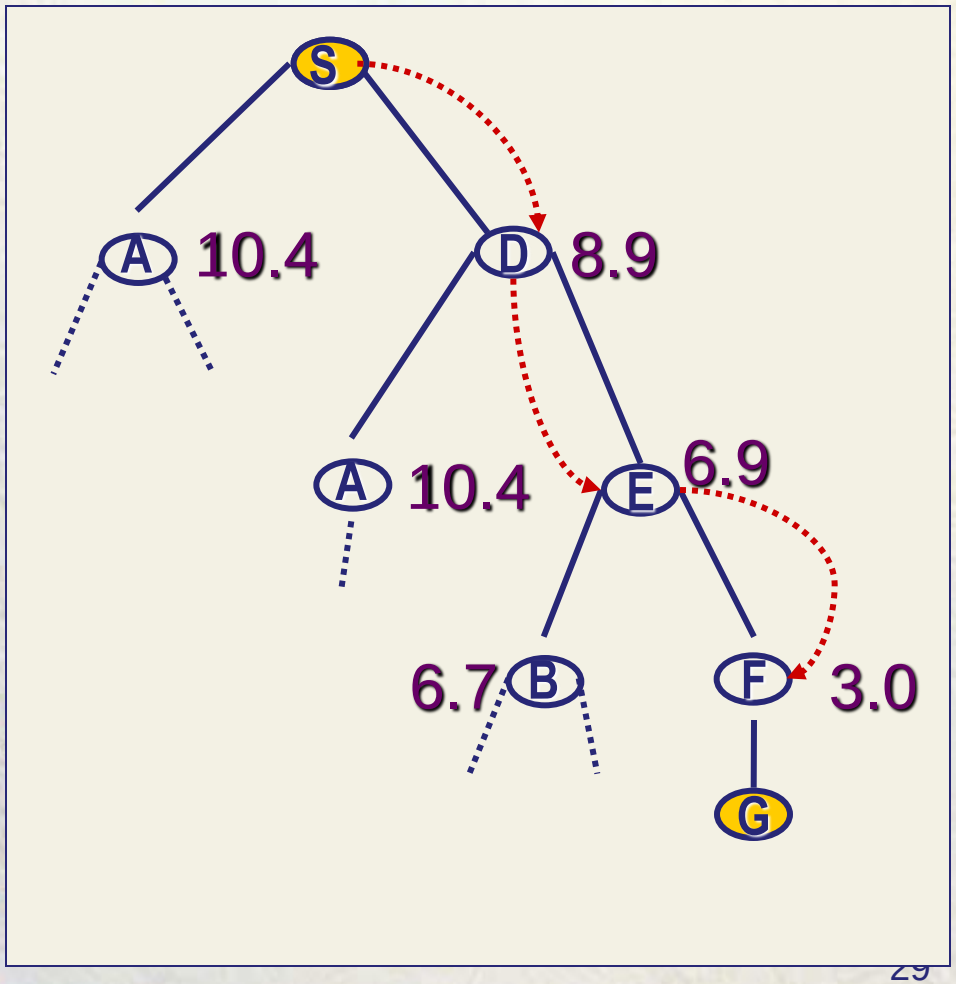
- Tìm kiếm leo đồi = tìm kiếm theo độ sâu + hàm đánh giá
- Khác với tìm kiếm theo độ sâu, ta phát triển một đỉnh u thì bước tiếp theo, ta chọn trong số các đỉnh con của u, đỉnh có nhiều hứa hẹn nhất để phát triển
- Đỉnh hứa hẹn nhất là đỉnh có hàm đánh giá nhỏ nhất

◎ Define f(T) = the straight-line distance from T to G



Tìm kiếm leo đồi

◎ Example: using the straight-line distance:

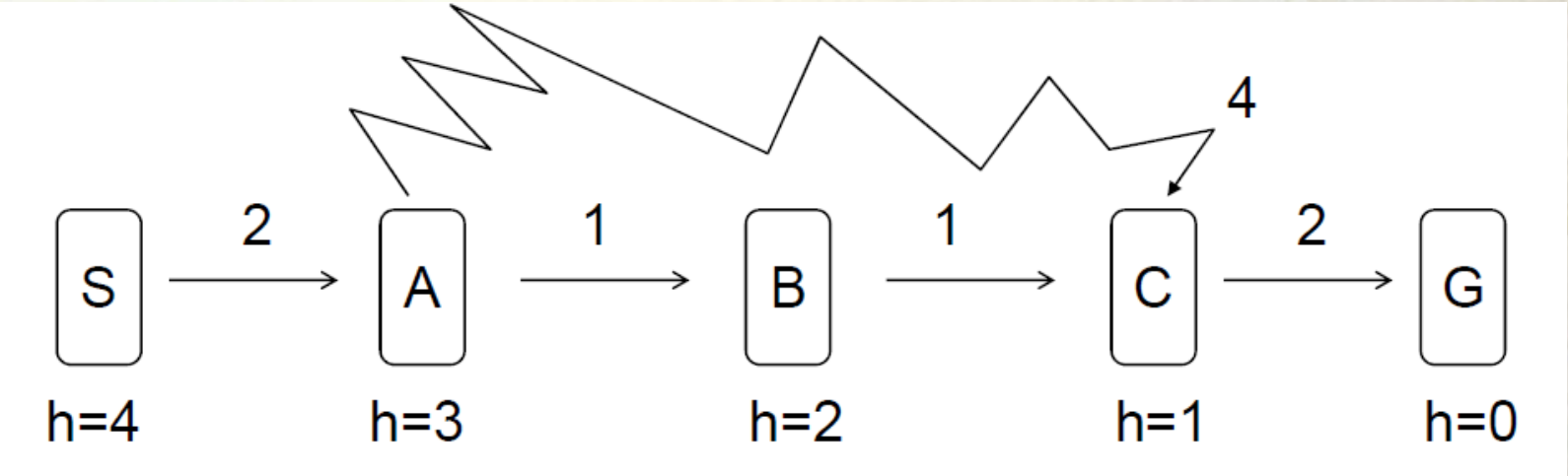


- Perform depth-first, BUT:
- instead of left-to-right selection,
- FIRST select the child with the best heuristic value

Thuật toán tìm kiếm leo đồi

1. Khởi tạo danh sách **L** chỉ chứa trạng thái ban đầu;
2. **Loop do**
   - 2.1. **if** L rỗng then
     { thông báo thất bại; **stop**};
   - 2.2. Loại trạng thái u ở đầu danh sách **L**;
   - 2.3. **if** u là trạng thái kết thúc **then**
     {thông báo thành công, **stop**};
   - 2.4. **for** mỗi trạng thái v kề u do đặt v vào **L1**;
   - 2.5. Sắp xếp **L1** theo thứ tự tăng dần của hàm đánh giá sao cho trạng thái tốt nhất ở đầu danh sách **L1**
   - 2.6. Chuyển danh sách **L1** vào đầu danh sách **L** sao cho đầu danh sách **L1** trở thành đầu **L**

Hãy Xem “tốt nhất” ngớ ngẩn thê nào?



- Tìm kiếm tốt nhất đầu tiên có đảm bảo tìm kiếm đường tối ưu?
- Chúng ta làm gì để tránh lỗi này?

Tìm kiếm A* (A* Search)

Tìm kiếm A*

- A* Search uses evaluation function f(n) = g(n) + h(n)
  - g(n): cost from initial node to node n
  - h(n): estimated cost of cheapest path from n to goal.
  - f(n): estimated total cost of cheapest solution through n.
- Tìm kiếm tốt nhất đầu tiên chọn h(n) nhỏ nhất
  - Hiệu quả nhưng không tối ưu
- Tìm kiếm Uniform-cost chọn g(n) nhỏ nhất
  - Tối ưu nhưng không hiệu quả

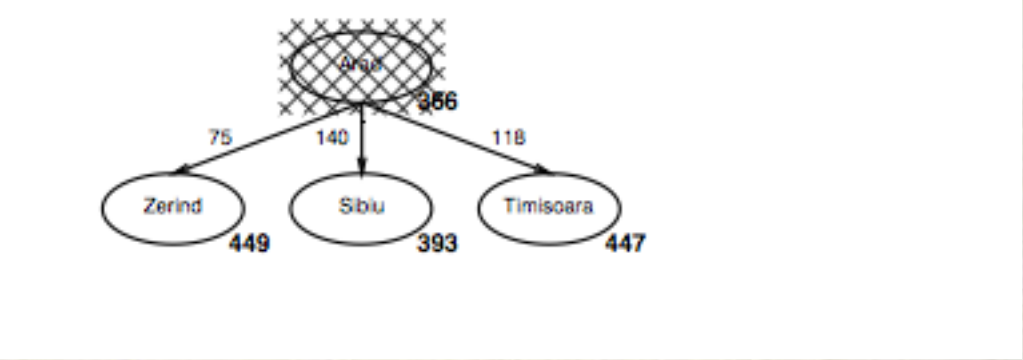
Tìm kiếm A*

- Tìm kiếm A* là làm cho f(n) = g(n) + h(n) nhỏ nhất.
  - Ý tưởng: preserve efficiency of Greedy Search but avoid expanding path that are already expensive
- Câu hỏi: Is A* search optimal and complete?
- Yes! Provided h(n) is *admissible*- nếu h(n) không vượt quá giá trị thực tế từ trạng thái n tới đích.
- http://theory.stanford.edu/~amitp/GameProgramming/AStarComparison.html

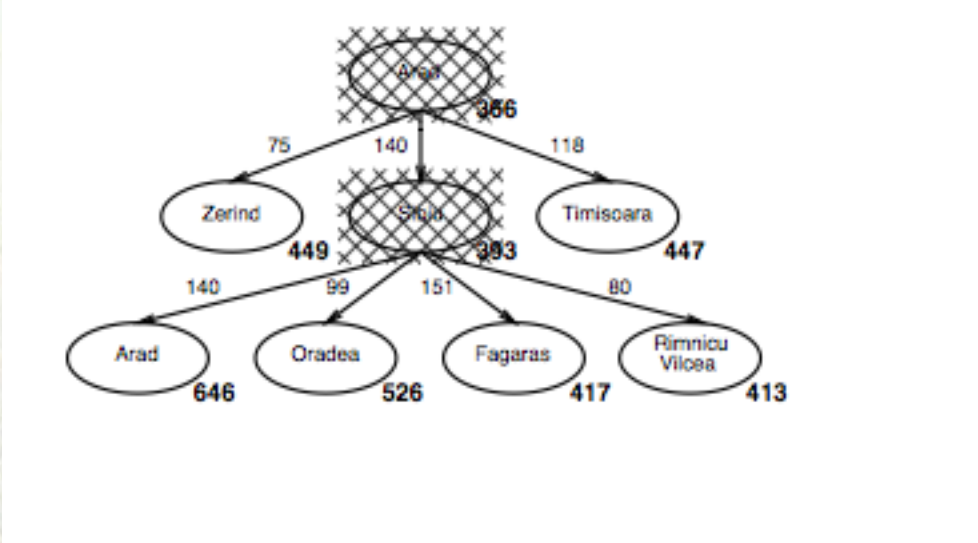
A* Search Example



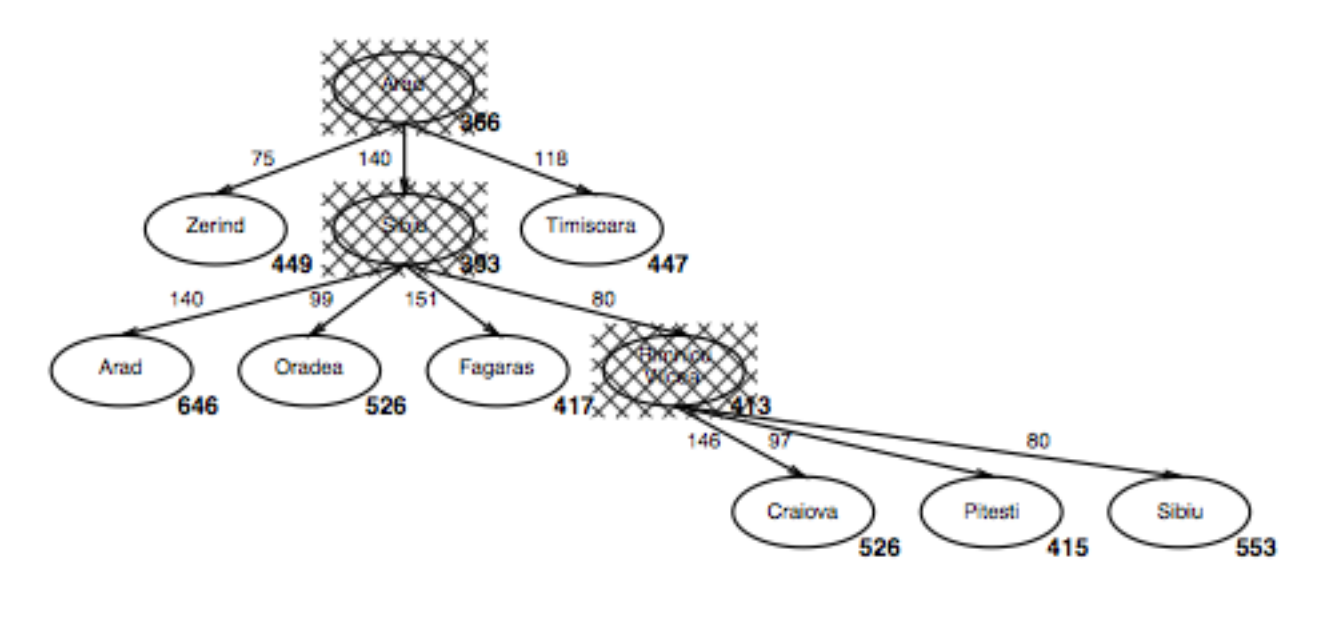
A* Search Example



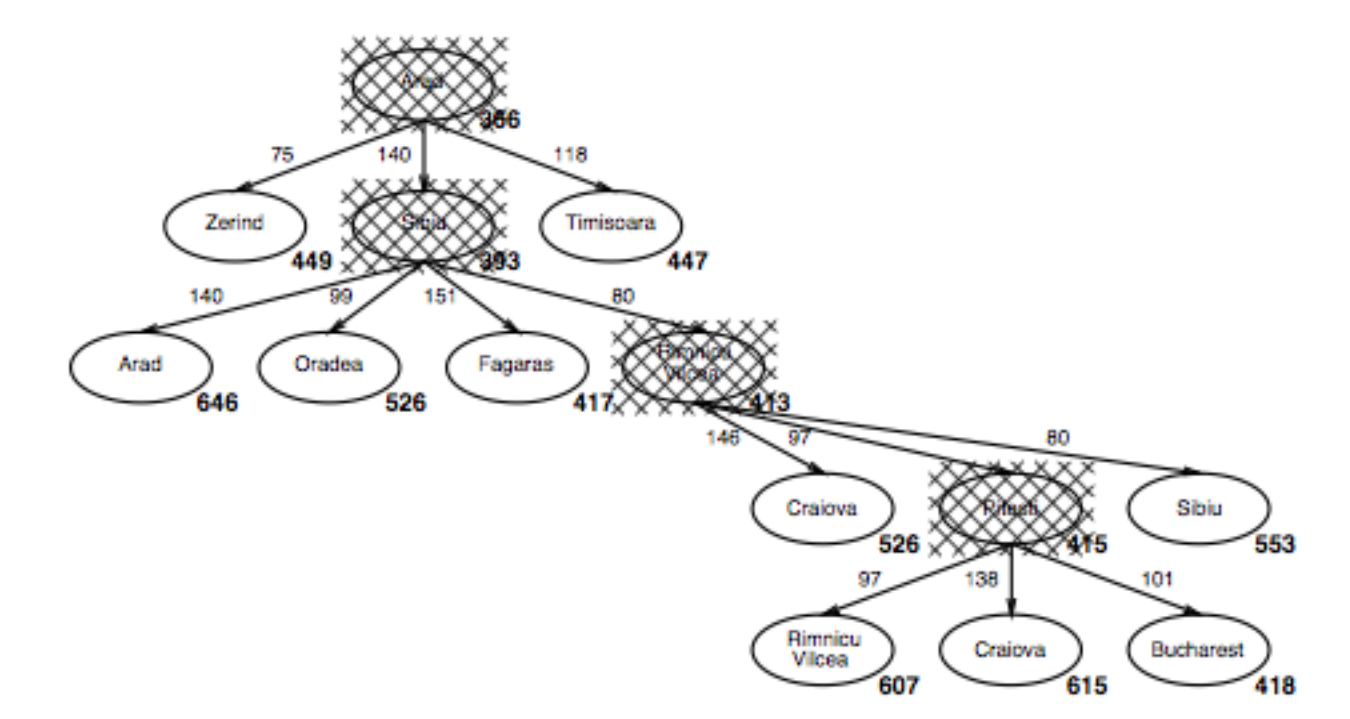
Ví dụ về tìm kiếm A*



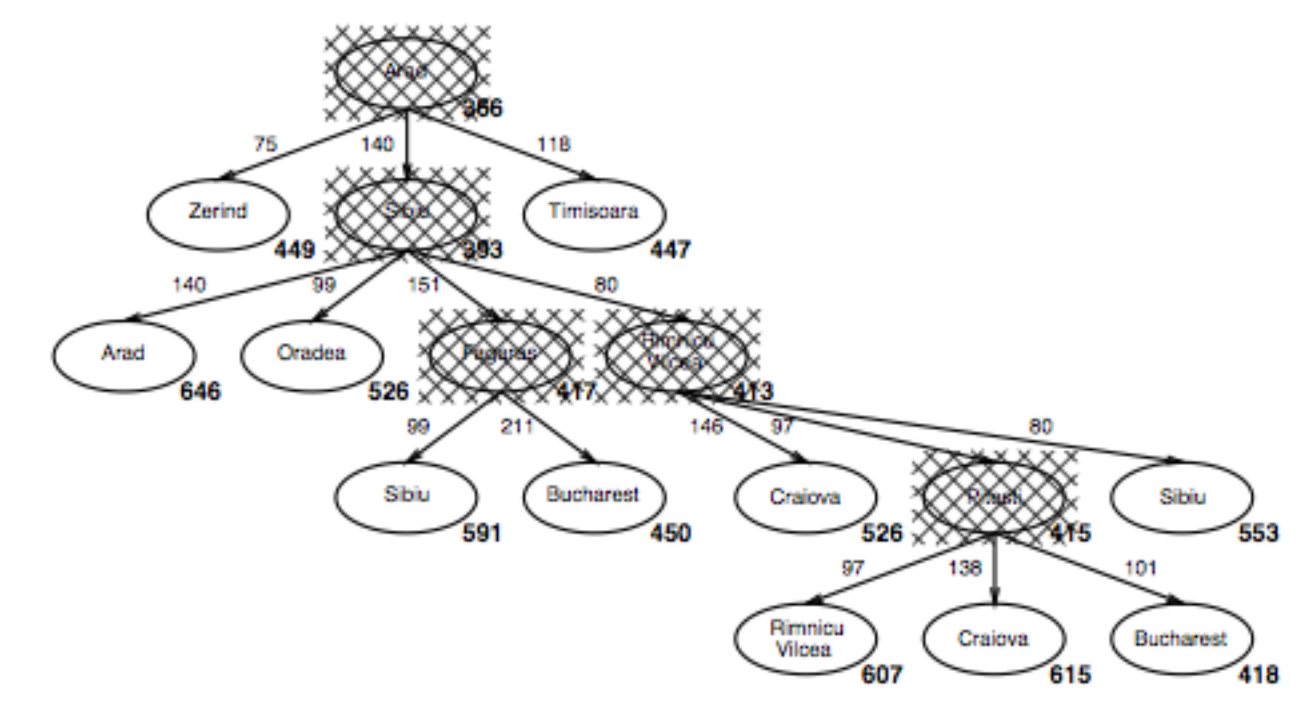
Ví dụ với tìm kiếm A*



A* Search Example



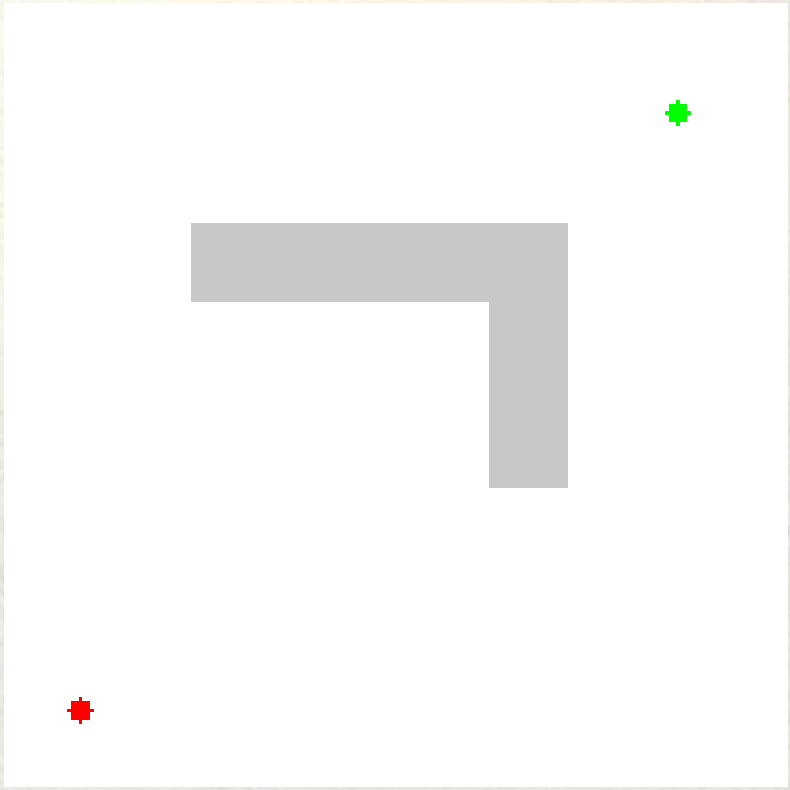
A* Search Example



Thuật toán A*

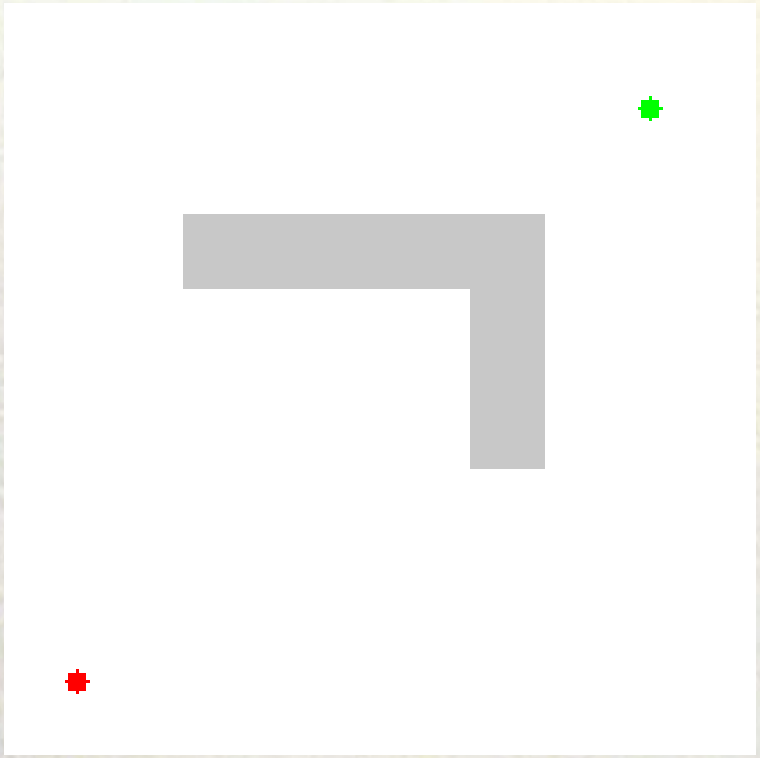
1. Khởi tạo danh sách L chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu danh sách L;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {g(v) := g(u) + k(u,v);
     f(v) := g(v) + h(v);
     Đặt v vào danh sách L sao cho L được sắp xếp thep thứ tự tốt đến xấu của hàm đánh giá f;}

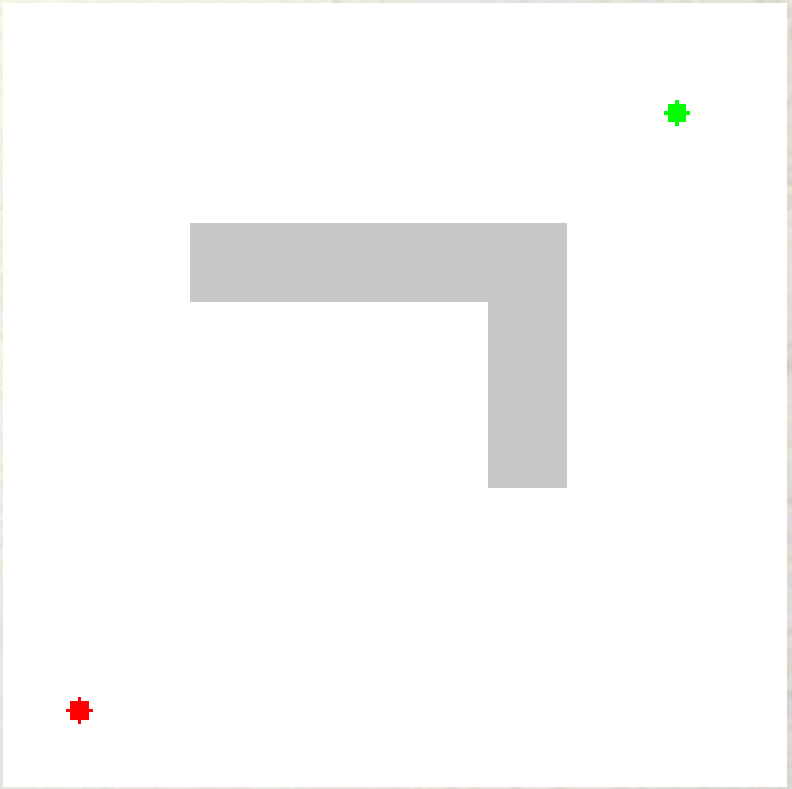
Another example



Source: Wikipedia

Uniform cost search vs. A* search





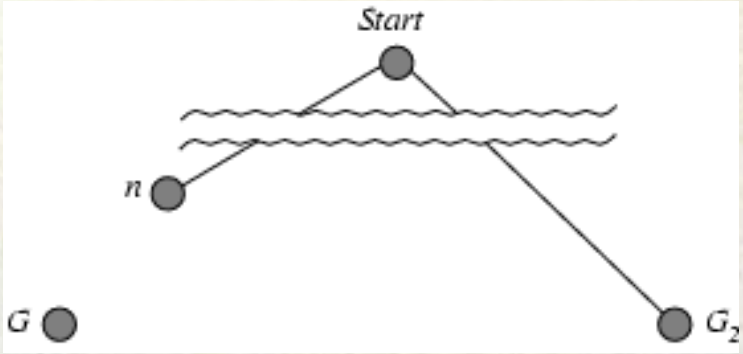
Source: Wikipedia

Admisible Heuristics

- Hàm đánh giá chấp nhận được
- Hàm đánh giá h(n) là chấp nhận được nếu h(n) luôn "lạc quan": không bao giờ đánh giá cao hơn giá trị thực.
- Nếu h(n) chấp nhận được thì A* luôn cho nghiệm tối ưu.

Optimality of A* (proof)

- Suppose some suboptimal goal G_2 has been generated and is in the fringe. Let *n* be an unexpanded node in the fringe such that *n* is on a shortest path to an optimal goal *G*.



- $f(G_2)$ > $f(G)$ from above
- $h(n)$ ≤ $h^*(n)$ since h is admissible
- $g(n) + h(n)$ ≤ $g(n) + h^*(n)$
- $f(n)$ ≤ $f(G)$
-

Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion

Optimality of A* Search

- Since $f(G_2) > f(n)$, A* will never select G_2 for expansion.
- The suboptimal goal node G_2 may be *generated*, but it will never be *expanded*.
- In other words, even after a goal node has been generated, A* will keep searching so long as there is a possibility of finding a shorter solution.
- Once a goal node is selected for expansion, we know it must be optimal, so we can terminate the search.

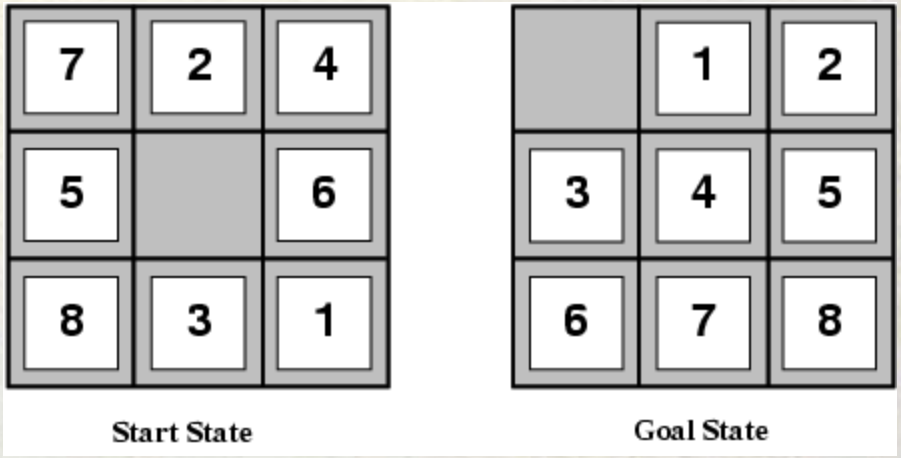
Properties of A* search

- Complete? Yes (unless there are infinitely many nodes with f ≤ *f(G)*)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes
-

Admissible heuristics (Hàm đánh giá chấp nhận được)

Bài toán 8 số (8-puzzle):

- $h_1(n)$ = số quân nằm không đúng vị trí
- $h_2(n)$ = Tổng số khoảng cách vị trí của các quân và vị trí đích của chúng

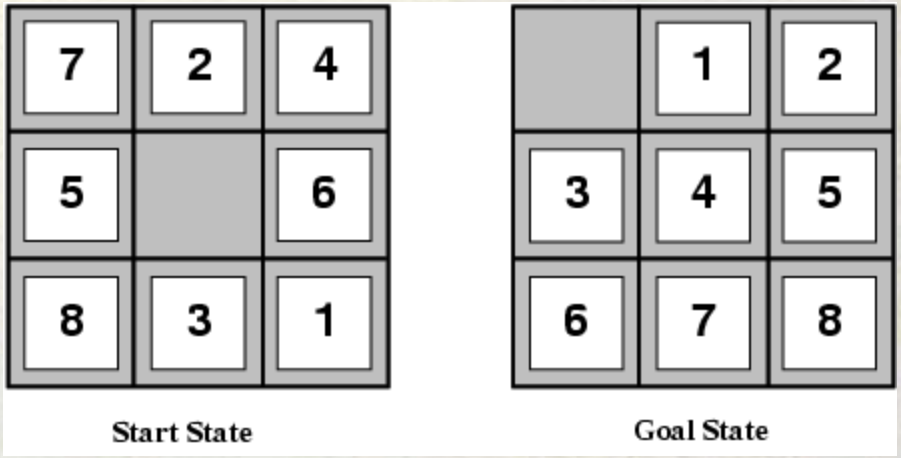


- $h_1(S) = ?$
- $h_2(S) = ?$
-

Hàm đánh giá chấp nhận được

Bài toán 8 số (8-puzzle):

- $h_1(n)$ = số quân nằm không đúng vị trí
- $h_2(n)$ = Tổng số khoảng cách vị trí của các quân và vị trí đích của chúng



- $h_1(S) = ?$ 8
- $h_2(S) = ?$ 3+1+2+2+2+3+3+2 = 18

Dominance (Tính áp đảo)

- If $h_2(n) \geq h_1(n)$ for all n (both admissible)
- then h_2 dominates h_1 (h_2 *áp đảo* h_1)
- h_2 is better for search
-
- Typical search costs (average number of nodes expanded):

- *Độ sâu d=12*

  IDS = 3,644,035 nodes
  $A^*(h_1)$ = 227 nodes
  $A^*(h_2)$ = 73 nodes
- *Độ sâu d=24*

  IDS = too many nodes ~ $54 * 10^9$ nodes
  $A^*(h_1)$ = 39,135 nodes
  $A^*(h_2)$ = 1,641 nodes

Cách tìm Hàm đánh giá chấp nhận được

- Giảm bớt ràng buộc.
- Một bài toán với ít rằng buộc hơn trong các hành động được gọi là bài toán nới lỏng (relaxed problem)
- Giá của giải pháp tối ưu cho bài toán nới lỏng là hàm đánh giá chấp nhận được của bài toán ban đầu

- Nếu luật của bài toán 8 số là nới lỏng để các quân có thể di chuyển tùy ý (bất cứ chỗ nào) thì $h_1(n)$ *cho giải pháp ngắn nhất*
- Nếu luật của bài toán 8 số là nới lỏng để các quân có thể di chuyển bất kì ô vuông liền kề, thì $h_2(n)$ *cho giải pháp ngắn nhất*
-

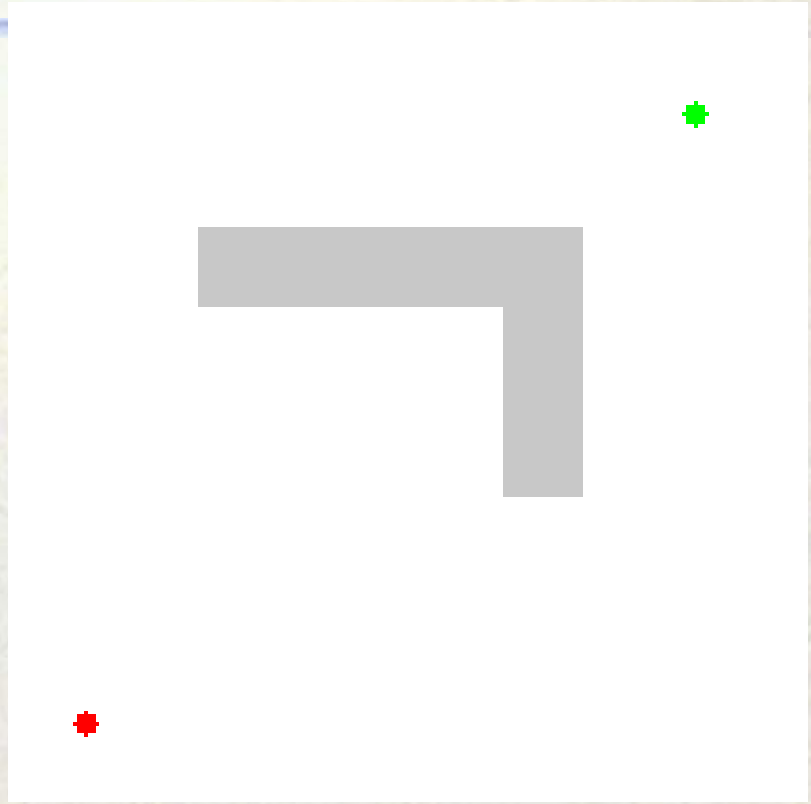
Composite Heuristic Functions

- Let $h_1, h_2, .., h_m$ be admissible heuristics for a given task.
- Define the composite heuristic (hàm đánh giá kết hợp):
  - $h(n) = \max(h_1(n), h_2(n), \ldots, h_m(n))$.
- h là hàm chấp nhận được
- h áp đảo $h_1, h_2, \ldots, h_m$

Weighted A* search

- **Idea:** speed up search at the expense of optimality
- Take an admissible heuristic, “inflate” it by a multiple $\alpha > 1$, and then perform A* search as usual
- Fewer nodes tend to get expanded, but the resulting solution may be suboptimal (its cost will be at most α times the cost of the optimal solution)

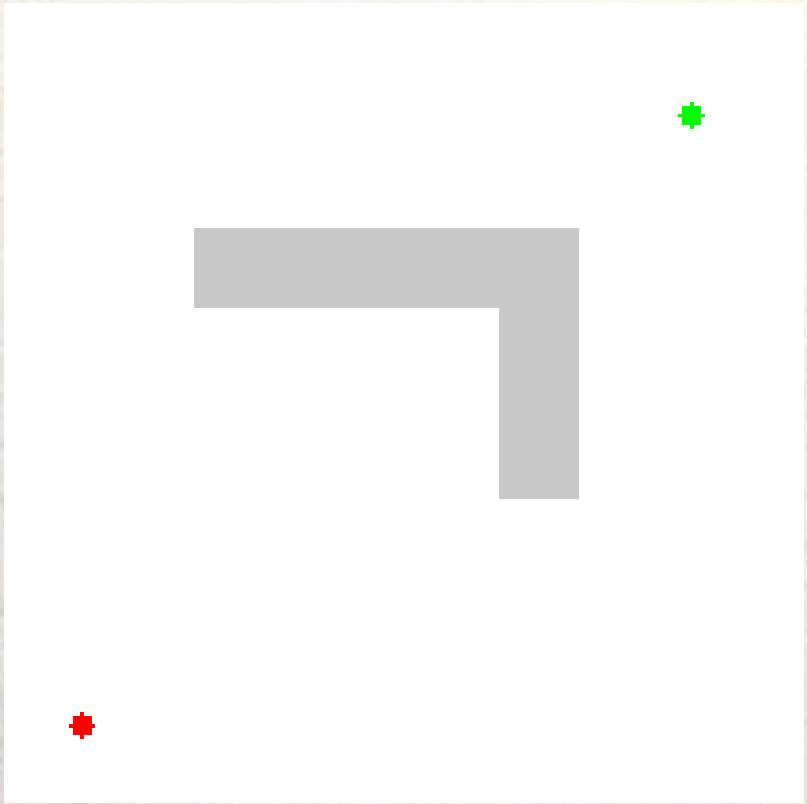
Example of weighted A* search

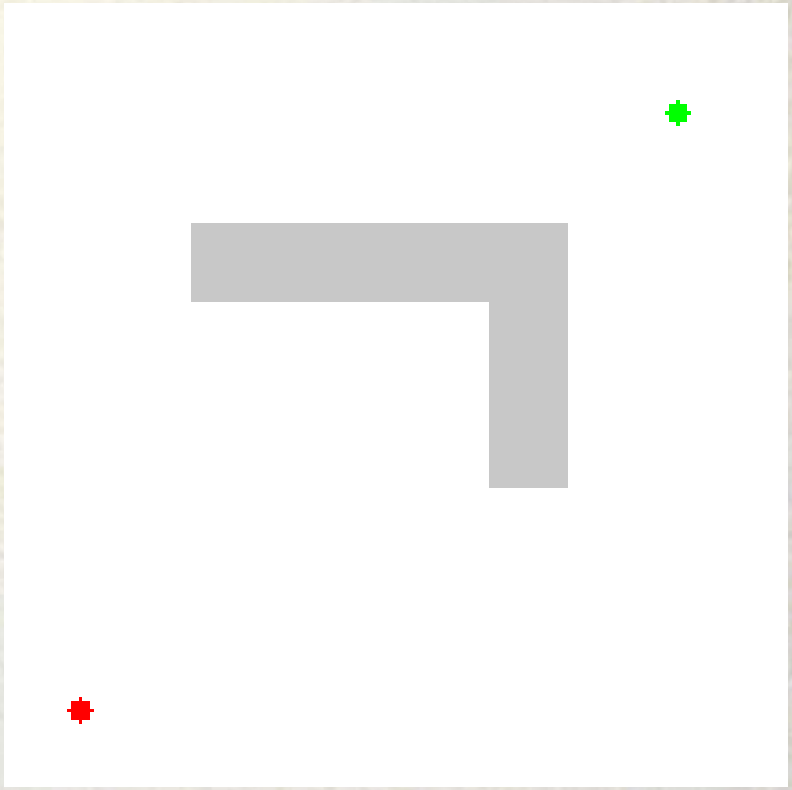


Heuristic: 5 * Euclidean distance from goal

Source: Wikipedia

Example of weighted A* search





Heuristic: 5 * Euclidean distance from goal

Source: Wikipedia

Compare: Exact A*

Cải tiến thuật toán A* bằng nhánh cận

1. Khởi tạo danh sách L chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu danh sách L;
   - **2.3' If L bao gồm đường đi P kết thúc tại trạng thái X với giá là cost_P, và đường đi Q bao gồm X với giá là cost_Q và cost_P >= cost_Q then**
     { Xóa P khỏi L}
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {g(v) := g(u) + k(u,v);
     f(v) := g(v) + h(v);
     Đặt v vào danh sách L sao cho L được sắp xếp thep thứ tự tốt đến xấu của hàm đánh giá f;}

Tài liệu tham khảo

- Trí Tuệ Nhân Tạo, Đinh Mạnh Tường
- Artificial Intelligence, A modern Approach. Chapter 4.

Tìm đường đi ngắn nhất từ S-> G bằng thuật toán A*

